

A Safety Profile Verifying Class Initialization

Document #: P3402R1
Date: 2024-10-11
Project: Programming Language C++
Audience: SG23
Reply-to: Marc-André Laverdière, Black Duck Software
<marc-andre.laverdiere@blackduck.com>
Christopher Lapkowski, Black Duck Software
<redacted@blackduck.com>
Charles-Henri Gros, Black Duck Software
<redacted@blackduck.com>

Contents

- 1 Abstract
- 2 Introduction
 - 2.1 Industry Demand
 - 2.2 Profile Guarantees
- 3 Definitions
- 4 Exemptions
 - 4.1 Exemption by Type
 - 4.2 Exemption by Attribute
- 5 Constraints on Static Data Members
- 6 Verification of Constructors
- 7 Templates
- 8 Idioms to Consider
 - 8.1 Non-Constructor Delegating
 - 8.2 Discipline with `const&` and `const*` Parameters
 - 8.3 Non-Initializing Default Initialization
- 9 Divergences from [P3274R0]
- 10 Future Work

- [11 Conclusion](#)
- [12 Straw Polls](#)
- [13 References](#)

§ 1 Abstract

We propose an attribute that specifies that every data member that belongs to a verified class has all its data members initialized to determinate values, assuming that the data used for construction is itself initialized properly.

This safety profile restricts what kind of code a constructor can have. Existing code bases are likely to violate these constraints, and thus this feature is an opt-in.

§ 2 Introduction

There is a growing push towards greater memory safety and memory-safe languages. While C++ is not memory-safe, it is desirable to specify and opt-in mechanism allowing a subset of C++ features that would result in memory safe programs. This has been termed ‘profiles’([\[P3274R0\]](#)), and would be specified at the TU level using an attribute. In this paper, we propose `[[Profiles::enable(initialization)]]`.

§ 2.1 Industry Demand

Industry compliance standards, such as CERT C++ [\[CERT\]](#), forbid access of uninitialized memory (Rule EXP53-CPP). While they imply complete initialization, they do not specify how a good constructor would achieve that objective.

However, the automobile safety industry desires fully initialized class objects. As part of the The MISRA C++ standard [\[MISRA\]](#), there are two rules that specifically advise proper initialization of class objects.

MISRA C++2023 Rule 15.1.2 “All constructors of class should explicitly initialize all of its virtual base classes and immediate base classes”

MISRA C++2023 Rule 15.1.4 “All direct, non-static data members of a class should be initialized before the class object is accessible”

§ 2.2 Profile Guarantees

All classes under the purview of the profile attribute will have the guarantee that all its data members are properly initialized once the object is constructed, assuming that the data used for construction is itself initialized properly. In the case of a class that inherits from one or more classes, all its base classes must be compliant with this profile.

In this paper, we give examples with the profile attribute attached to specific classes. We do so to make it clear which classes are verified classes and which ones aren't, since some examples have a mix of them.

Example:

```
struct [[Profiles::enable(initialization)]] parent1 {
    int i;
    parent1() = default; //non-compliant, i is default initialized
};
struct [[Profiles::enable(initialization)]] child1 : public parent {
    int j;
    child1() : parent1(), j(42) {} //child is compliant, but parent isn't
}

//Not a verified class, but would be compliant if it were
struct parent2 {
    int i = 0;
    parent2() = default;
};
struct [[Profiles::enable(initialization)]] child2 : public parent2 {
    int j;
    child2() : parent2(), j(42) {} //child is not compliant, because parent2
                                is not a verified class
}
```

§ 3 Definitions

A *verified class* is a class that is affected by the profile attribute or a POD.

An *object parameter* is either the `this` pointer or an explicit object parameter ([`dcl.fct`]).

A *verified data member* is a data member that is not exempted from verification.

Acceptable inputs are:

1. The non-exempt transitive closure of verified data members
2. The non-exempt transitive closure of the parameters
3. Manifestly constant-evaluated expressions ([`expr.const`])
4. Object Parameters
5. Arithmetic operations whose operands are the above

The *non-exempt transitive closure* of X means the set of symbols that are reachable from X using the dot and arrow operators, and which are not exempt from verification.

§ 4 Exemptions

Some data members are exempt from verification, either due to intrinsic properties, or due to explicit opt-out from the developer.

§ 4.1 Exemption by Type

[P3274R0] mentions that performance critical applications won't initialize output buffers at first and mentions a few possibilities: "suppression, an uninitialized attribute, and/or by specific uninitialized types."

This profile allows a pointer to be initialized to any value, whether that's a `nullptr`, the return value of a `new`, or even a hardcoded address. It does not require that the memory space pointed to by the pointer is set to any value. This is an allowance for systems programming, which sometimes have buffers pointing to hardcoded addresses, which are used for interacting with devices. We therefore exempt dynamically allocated memory from initialization in this profile. Note that this profile prohibits spreading this uninitialized memory to verified non-static data members through requirement `[only.acceptable.in]`

A cleaner solution for uninitialized memory for the purpose of buffers, would be to use a specialized type. We envision a class named `std::RawBuffer<T>`, which would record which regions of the buffer have been written previously, and prohibit reads outside of that region. This class would have a use beyond this profile, making it a more generic solution.

We leave the general question of pointer safety to `[[Profiles::enable(Pointers)]]`.

§ 4.2 Exemption by Attribute

Developers could exempt specific data members from verification using the `[[indeterminate]]` attribute from [P2795R5].

```
struct [[Profiles::enable(initialization)]] HighPerformance {  
    std::byte* buf [[indeterminate]];  
    int sz = -1;  
    void fill(/*...*/);  
};
```

§ 5 Constraints on Static Data Members

The static data members (`[class.static.data]`) in a verified class are allowed in this profile, but, as long they are initialized solely using constant or zero initialization. This includes `constexpr` static data members.

Static data members have either static storage duration (`[basic.stc.static]`) or thread storage duration (`[basic.stc.thread]`). They are guaranteed to be initialized with constant initialization (`[basic.start.static]`). However, they can be reassigned during dynamic initialization (`[basic.start.dynamic]`).

Dynamic initialization can lead to subtle bugs, such as:

- Circular dependencies
- Use of statically-initialized values before they are dynamically-initialized.
- Initialization order issues when dealing with symbols variables defined in other translation units.
- Assignment of uninitialized values

We illustrate how uninitialized memory can affect static data members with dynamic initialization below.

```
int randomInt() {
    int therandomint;
    return therandomint;
}
struct [[Profiles::enable(initialization)]] WithStaticUninit1 {
    WithStaticUninit1() = default; //Not a POD
    static int thestatic;
};
int WithStaticUninit1::thestatic = randomInt(); //non-compliant

struct [[Profiles::enable(initialization)]] GetsCorrupted {
    GetsCorrupted() : thefield(0) {} //compliant
    int thefield;
};

struct [[Profiles::enable(initialization)]] Wrapper {
    Wrapper() = default; //Not a POD
    static GetsCorrupted wrapped;
};
GetsCorrupted corruptingFactory() {
    GetsCorrupted ret{}; //All initialized, good
    ret.thefield = randomInt(); //Now, some uninitialized memory snuck in
    return ret;
}
GetsCorrupted Wrapper::wrapped = corruptingFactory(); //Non-compliant
```

§ 6 Verification of Constructors

All constructors of a verified class must satisfy the following properties:

- [base.are.verified] All base classes must be verified classes.
- [base.init.first] All base classes must be initialized before any of their non-static data members are read.
- [delegating.ctor] A delegating constructor is compliant if the constructor it delegates to is also compliant and it respects the other rules in this section.
- [init.before.read] All verified data members must be initialized before being read.

- [init.all.paths] All verified data members must be initialized on all paths in the constructor.
- [only.acceptable.in] Only acceptable inputs can be assigned to verified data members and static data members of verified classes.
- [no.args] Acceptable inputs cannot be passed by pointer or reference to function calls, except to constructors.
- [effective.const] Except for move constructors, the constructor must not modify the state of acceptable inputs.
- [no.addrof] The address of acceptable inputs cannot be obtained.
- [no.capture] Verified inputs cannot be captured by reference in a lambda.

A data member is considered read whenever it is present in the function, except when:

- It is an unevaluated operand ([`expr.context`])
- It is part of a discarded statement ([`stmt.if`])
- It is the left operand of an assignment ([`expr.ass`])

This definition implies the following:

- The return value of non-manifestly constant-evaluated function calls may never be assigned to a data member, either directly or indirectly.
- A verified class' non-static data members are either verified classes or PODs. Note that PODs don't have constructors to call, even though the syntax used may give that impression.
- A nonconforming constructor would bring the rejection of its class only, and not of its subclasses. This decision is intended to reduce the noisiness that would come from a faulty constructor at the top of a very large class hierarchy.
- An initialized pointer can point to an uninitialized memory region (e.g. `buf(new char[BUF_SIZE])`) and be compliant. We discuss this [above](#).
- Default initialization of data members for some types mean that no initialization is performed ([`dcl.init`]). Constructors that use this category of default initialization are non compliant.
- Calls to `operator*`, `operator->` and `operator->*` are not allowed.
- The use of operators (e.g. `*v`, `&v`, `v[]`) to acceptable inputs is not allowed.
- The constructor may execute any code that does not affect verified data members.
- Verified data members may never be initialized using an array subscript.

We restrict function calls because we want to keep the analysis intraprocedural. Initialization that occurs in a member function, or occurs from a function's return value, or

verifying that arguments are not tampered with, would require interprocedural analysis. Keeping the analysis intraprocedural would facilitate adoption.

Examples:

```
struct [[Profiles::enable(initialization)]] clazz1 {
    int i;
    int j;
    int z = 0;
    clazz1() {
        i = 123;
        if (nondet) {
            j = 456;
        }
        //non-compliant: [init.all.paths]
    }
};
```

```
struct [[Profiles::enable(initialization)]] clazz2 {
    int i;
    int j;
    clazz2() : i(j), j(42) {} //non-compliant: [init.before.read]
};
```

```
struct [[Profiles::enable(initialization)]] clazz3 {
    int i;
    int j;
    clazz3() { //compliant, but bad form
        this->i = 0;
        this->j = 42;
    }
};
```

```
struct pod {
    int i;
    int j;
};

struct [[Profiles::enable(initialization)]] clazz4 {
    pod p;
    clazz4() = default; //non-compliant: non-initializing default
                        //initialization [init.all.paths]
};

struct [[Profiles::enable(initialization)]] clazz5 {
    pod p{};
    clazz5() = default; //compliant, p is value-initialized
}

struct [[Profiles::enable(initialization)]] clazz6 {
    pod podFactory() {
        pod p; // non-initializing default initialization
        return p;
    }
    clazz6() : p(podFactory()) {} //non-compliant: [only.acceptable.in]
}
```

```
struct [[Profiles::enable(initialization)]] clazz7 {
    int i;
    int j;
    clazz7(int i) : i(i), j() {} //compliant, j is value-initialized
};
```

```
struct [[Profiles::enable(initialization)]] clazz8 {
    int i;
    int j;
    utility_function() const;
    clazz8(int i) : i(i), j() {
        utility_function(); //non-compliant: [no.args]
    }
};
```

```
struct [[Profiles::enable(initialization)]] clazz9 {
    int i;
    int j;
    void mutating(int&) const;
    clazz9(int i) : i(i), j() {
        mutating(j); //non-compliant: [no.args]
    }
};
```

```
struct [[Profiles::enable(initialization)]] clazz10 {
    int i;
    int j;
    void mutating();
    clazz10(int i) : i(i), j() {
        mutating(); //non-compliant: [no.args]
    }
};
```

```
struct [[Profiles::enable(initialization)]] clazz11 {
    int i;
    int j;
    void mutating() const {
        int uninit;
        const_cast<clazz11*>(this)->j = uninit;
    }
    clazz11(int i) : i(i), j() {
        mutating(); //non-compliant: [no.args]
    }
};
```

```
struct [[Profiles::enable(initialization)]] clazz12 {
    std::byte* buf [[indeterminate]];
    size_t    buf_size;
    int i;
    clazz12() : i(std::to_integer<int>(buf[0])) {} //non-compliant: [only.acceptable.in]
};
```

```
struct [[Profiles::enable(initialization)]] clazz13 {
    static unsigned num_allocations;
    clazz13() {
```

```

    ++num_allocations; //compliant: [only.acceptable.in]
}
};
unsigned clazz13::num_allocations = 0;

```

Please note that this profile will mark some correct code as non-compliant. This is unavoidable.

```

struct [[Profiles::enable(initialization)]] clazz14 {
    std::byte* buf [[indeterminate]];
    size_t    buf_size;
    int i;
    clazz14(size_t sz) : buf_size(sz) {
        if (sz > 0) {
            buf = new std::byte[sz];
            std::fill(buf, buf + sz, std::byte{0});
            i = std::to_integer<int>(buf[0]); //safe, but non-compliant: [only.acceptable.in]
        } else {
            buf = nullptr;
            i = -1;
        }
    }
};

```

§ 7 Templates

In the case of templated classes, the property is verified during template instantiation.

```

class NotAnnotated{/**/};
class [[Profiles::enable(initialization)]] Annotated {/**/};

template<typename T>
class [[Profiles::enable(initialization)]] AnnotatedTemplate {
    T field = T();
};

void foo() {
    AnnotatedTemplate<NotAnnotated> nat {}; //non-compliant, calling the constructor to a non-verified class
    AnnotatedTemplate<Annotated> at {}; //compliant
}

```

§ 8 Idioms to Consider

§ 8.1 Non-Constructor Delegating

A recent SG23 mailing list discussion highlighted that delegating initialization to a non-constructor member function is idiomatic in C++. Supporting this idiom would make this profile more useful.

The bit of code that triggered the discussion is the following:

```
basic_string(const _CharT* __s, const _Alloc& __a = _Alloc())
: _M_dataplus(_M_local_data(), __a)
{
    //...
    _M_construct(__s, __end, forward_iterator_tag());
}
```

In this case, `_M_local_data()` returns a `const` pointer to a data member (`_M_local_buf`) and passes it to the `_M_dataplus` data member. The initialization then is done by `_M_construct`. This code would be reported as violating the safety profile as we specify it in this draft, since the constructor does not initialize `_M_dataplus` directly.

There are a few solutions to this problem:

1. Force developers to rewrite their code to use delegating constructors.
2. Use an on-demand interprocedural analysis that ensures that initialization happens on all paths in the callee.
3. A suggestion was to annotate arguments that must be initialized with `[[must_init]]`.

```
struct DelegatingInit {
    int member;
    DelegatingInit() {
        internal_init(&member);
    }
    internal_init([[must_init]] int* p);
}
```

This option is less intrusive than option 1, and would be simpler to verify than option 2, simply because the scope of the analysis becomes well-bounded. As such, it is worth considering.

Nonetheless, we consider it undesirable for the following reasons:

- Adding another attribute is undesirable.
- Only explicit parameters can be initialized this way. However, developers might like to have a general `initialize()` member function.
- Functions with the `[[must_init]]` attribute can delegate further to other `[[must_init]]` functions, this can lead to recursion.

§ 8.2 Discipline with `const&` and `const*` Parameters

While C++ allows to strip `const`-ness through casting, this practice is uncommon. It would be preferable to allow function calls in constructors pass acceptable inputs by `const` reference or `const` pointer. However, we need a way to ensure that the callees are do not use shenanigans allowing them to modify the state of acceptable inputs.

There are a few solutions to this problem:

1. Require that all member functions in verified classes enforce a variant of criterion [effective.const].
2. Add an attribute (e.g. `[[const_is_const]]`) at the function declaration that indicates to the analyzer that a variant of criterion [effective.const] must be verified for the function.
3. Require that all functions within the scope of the profile attribute be prohibited from using mechanisms that remove `const`-ness, unless they have the novel `[[unsafe]]` attribute.
4. The verifier could use an interprocedural analysis.

Regarding solution 1, there are edge cases with virtual member functions and we wouldn't allow verified classes' constructors to call non-member functions.

Regarding solution 2, we don't like adding additional attributes, which may hamper adoption. It does however keep the analysis simple.

Regarding solution 3, it widens the scope of the profile. While that is a good thing, it may be challenging to communicate exactly what the scope is to developers.

Regarding solution 4, it is the most obvious solution, but is restricted to cases where the call target can be determined accurately. It would also increase the complexity of the implementation and, in turn, risk to hamper adoption.

§ 8.3 Non-Initializing Default Initialization

Many restrictions stem from the fact that default initialization sometimes mean that no initialization is performed ([`dcl.init`]). Prohibiting its use in functions called from verified classes' constructors could be beneficial. However, it expands the scope of the profile.

§ 9 Divergences from [P3274R0]

This draft materially deviates from [P3274R0] in the following ways:

- We do not assume that all default constructors are initializing data members.
- We do not require that data members are initialized only once, since multiple assignments are not a correctness issue.
- We do not take position on implicit initialization and whether it is error-prone.
- We added the profile attribute at the class level, rather than at the level of the translation unit. As explained above, this is for making examples more communicative. Our proposal is orthogonal to the location of the attribute.

§ 10 Future Work

The proposed attribute could be misunderstood to mean that all variables in all the code in the scope of the attribute are properly initialized. This may be addressed in a future version of this profile, or in another profile.

We also observe that profiles imply a constraint on what types can be used in a template. This hints at a new concept. A future revision of this paper would explore this further.

§ 11 Conclusion

In this paper, we propose a safety profile that guarantees that any class affected by the profile attribute will have all its data members initialized, assuming that the data used for construction is itself initialized properly. The profile does not depend on the presence of specific modern C++ features and can thus be applied to legacy code bases.

§ 12 Straw Polls

Q1: Should an attribute be used to exempt data members from initialization?

Q2: Should specific types be used to exempt data members from initialization?

Q3: Should this profile handle initialization outside of constructors?

Q4: If this profile were to handle initialization outside of constructors, should the profile rely on an attribute on parameters that indicates what the function is responsible for initializing?

Q5: If this profile were to allow verified inputs to be passed by `const&` or `const*`, how would it ensure that they are not tampered with?

Q6: For a given scope of applicability (e.g. translation unit), should this profile prohibit the use of default initialization altogether? Or prohibit the use of default initialization leading to no initialization?

Q7: Given that this profile does not consider initialization at large, should we rename it to `class_initialization`?

§ 13 References

[CERT] SEI CERT. 2016. SEI CERT C++ Coding Standard.

<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046682>

[MISRA] The MISRA Consortium and Chris Tapp. 2023. MISRA C++:2023: Guidelines for the use of C++17 in critical systems.

<https://misra.org.uk/misra-cpp2023-released-including-hardcopy/>

[P2795R5] Thomas Köppe. 2024-03-22. Erroneous behaviour for uninitialized reads.

<https://wg21.link/p2795r5>

[P3274R0] Bjarne Stroustrup. 2024-05-10. A framework for Profiles development.

<https://wg21.link/p3274r0>